



SIMATS ENGINEERING
CO5-ASSESSMENT TOOL -13
Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



Student Name: TALARI VISHNUVARDHN

Reg. No.: 192572091

Course Code: CSA1511

Slot: B

Course Name: CLOUD COMPUTING AND BIG DATA ANALYTICS

Course Faculty: Dr. Hemavathi R

1. Design an HDFS-Based Storage Solution for a Media Platform

Answer:

A media platform stores a large amount of data such as **videos, images, audio files, and documents**. An HDFS-based storage solution should provide **high availability above 99.9%**, while the **replication factor should not exceed 3** and storage costs should remain efficient.

Proposed Architecture:

Media Users / Applications



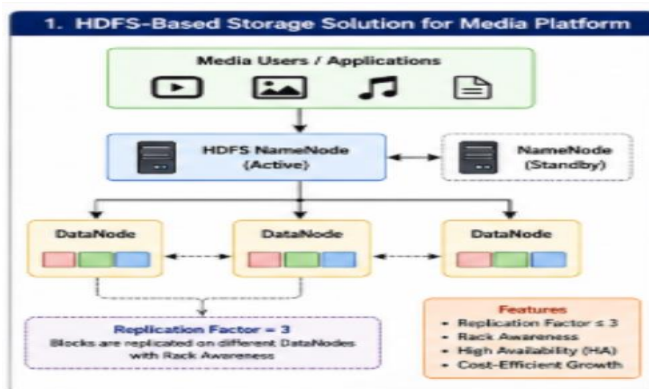
HDFS NameNode



Multiple DataNodes



Replicated Media Data Blocks



Design Components:

1. HDFS Storage

All media files are stored in the Hadoop Distributed File System (HDFS). Large files are divided into smaller blocks and distributed across different DataNodes.

2. Replication Factor = 3

Each data block is stored in a maximum of three different locations. This satisfies the given constraint and provides fault tolerance.

3. Rack Awareness

The replicas are placed across different racks. If one rack fails, another copy of the data can still be accessed from another rack.

4. NameNode High Availability

An Active NameNode and Standby NameNode can be used. If the Active NameNode fails, the Standby NameNode takes over.

5. Cost-Efficient Storage Growth

Storage can be increased gradually by adding new DataNodes when the amount of media data increases. This avoids unnecessary infrastructure costs.

Justification:

The proposed HDFS solution provides high availability because multiple copies of data are stored on different DataNodes. The replication factor is limited to **3**, satisfying the given constraint. Rack awareness protects against hardware and rack failures. NameNode High Availability prevents a single point of failure. Therefore, the system can achieve **high data availability above 99.9%** while maintaining cost-efficient storage.

2. Design a MapReduce Workflow for Processing Log Data Within 20 Minutes

Answer:

The objective is to process a large amount of log data within a **20-minute batch window** using a limited number of cluster nodes. Therefore, the MapReduce job must use efficient parallel processing and proper task distribution.

Proposed Workflow:

Log Files



HDFS Storage



Input Splits



Mapper Tasks



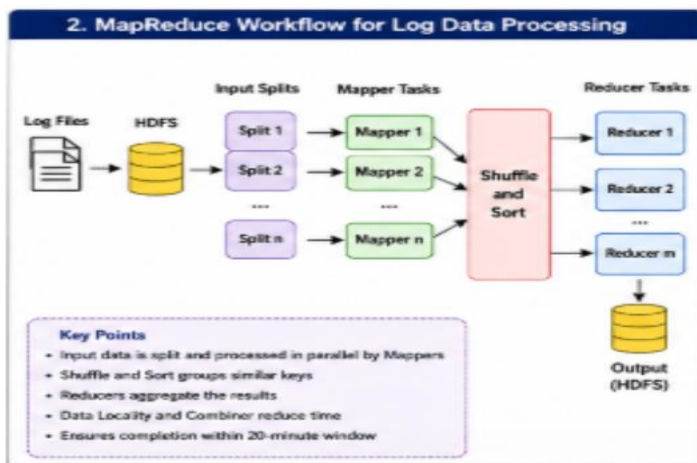
Shuffle and Sort



Reducer Tasks



Final Output



Working of the System:

1. Input Data Storage

The log files are first stored in HDFS. HDFS divides large log files into blocks.

2. Input Splitting

The input data is divided into multiple smaller input splits. Each split can be processed independently.

3. Mapper Tasks

Multiple Mapper tasks process different input splits in parallel. Each Mapper extracts useful information from the log files.

For example:

4. Shuffle and Sort

The intermediate output from all Mapper tasks is grouped according to similar keys.

5. Reducer Tasks

Reducer tasks combine and summarize the results received from the Mapper tasks.

6. Data Locality

Tasks should be executed close to the DataNode where the required data is stored. This reduces network communication.

7. Combiner Optimization

A Combiner can be used to reduce the amount of intermediate data transferred between Mapper and Reducer.

Justification:

Parallel Mapper execution helps process large log files quickly. Data locality reduces network delays, while the Combiner reduces data transfer during the shuffle phase. Proper distribution of Mapper and Reducer tasks allows the workflow to complete within the **20-minute batch window**, even when cluster resources are limited.

3. Design a YARN Scheduler Policy for ETL, Reporting, and Machine Learning Jobs

Answer:

A YARN-managed cluster must support **ETL jobs, reporting jobs, and machine learning jobs**. The scheduler should provide fairness while giving higher priority to ETL processes and recovering from node failures.

Proposed Architecture:

Users / Applications



YARN ResourceManager



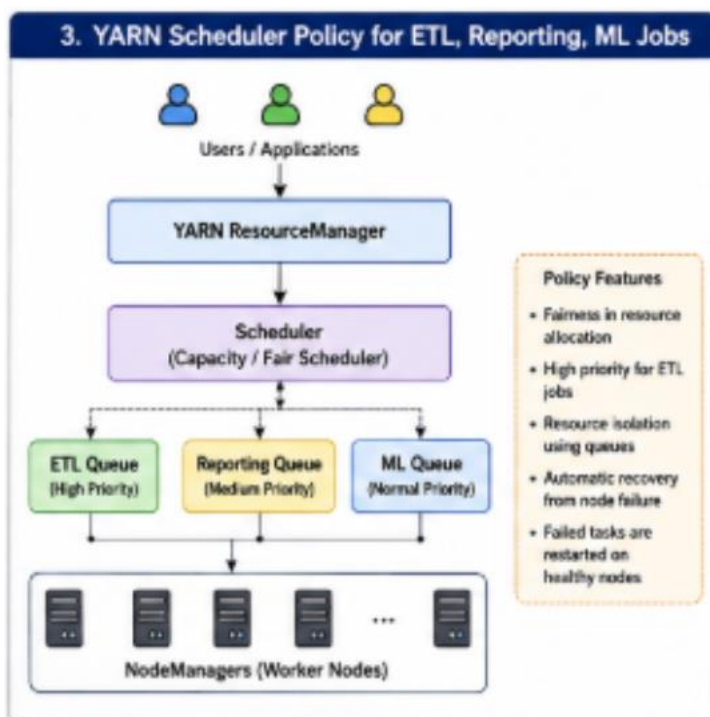
Scheduler



ETL Queue | Reporting Queue | ML Queue



NodeManagers



Scheduler Policy:

Job Type	Priority
ETL Jobs	High
Reporting Jobs	Medium
Machine Learning Jobs	Normal

Working:**1. Separate Queues**

Separate queues are created for ETL, reporting, and machine learning workloads.

2. Priority for ETL

ETL jobs are given higher priority because they prepare and transform the data required by other applications.

3. Fair Resource Allocation

The scheduler distributes available CPU and memory resources fairly among the different queues and users.

4. ResourceManager

The ResourceManager manages the overall cluster resources and allocates resources to applications.

5. NodeManager

Each worker node contains a NodeManager that monitors and manages resources on that node.

6. Failure Recovery

If a NodeManager or worker node fails, YARN detects the failure. The failed tasks can be restarted on another available node.

Justification:

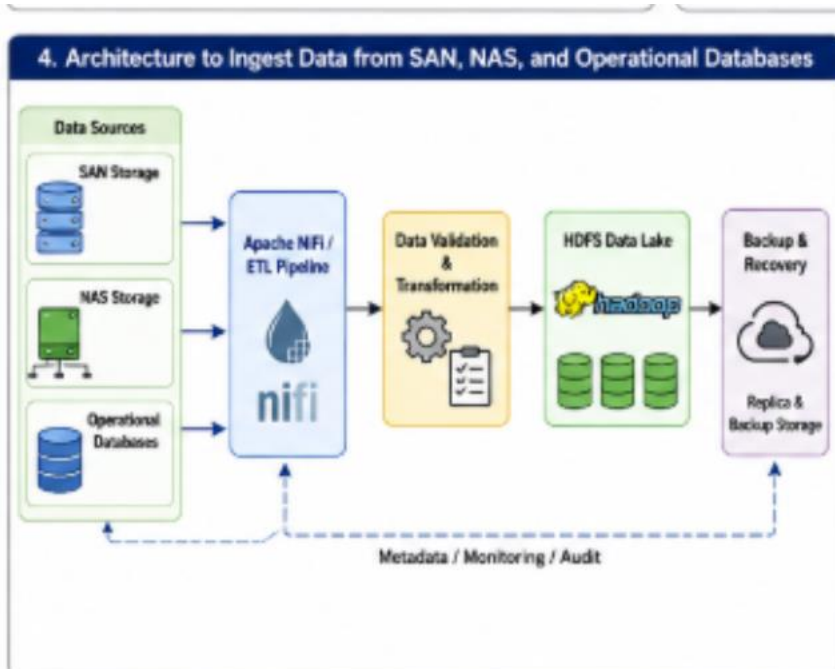
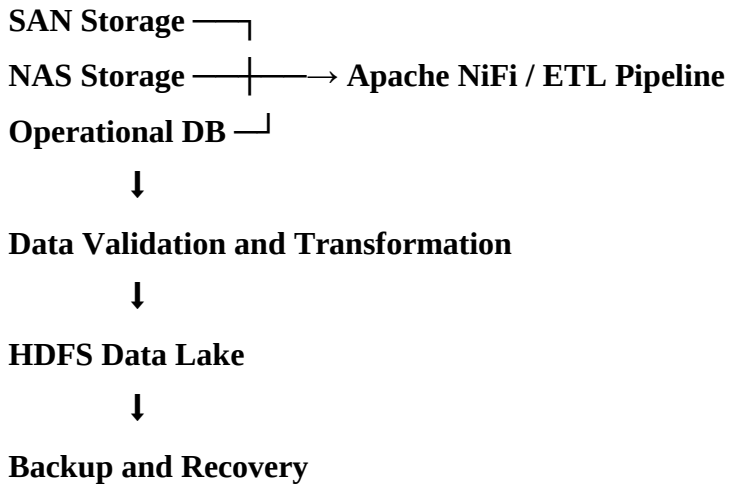
The proposed policy provides fairness by sharing resources among different workloads. ETL jobs receive higher priority to ensure that important data processing is completed first. Reporting and machine learning jobs receive resources based on availability. Node failure recovery ensures that applications can continue

4. Design an Architecture to Ingest Data from SAN, NAS, and Operational Databases

Answer:

An enterprise needs to collect data from **SAN, NAS, and operational databases** and store it in a big data platform. The architecture must provide reliable backup while minimizing unnecessary duplication.

Proposed Architecture:



Components:

1. SAN

SAN provides enterprise-level storage. Required data can be collected and transferred to the big data platform.

2. NAS

NAS contains shared files and unstructured data. These files can be ingested through the data pipeline.

3. Operational Databases

Data from databases can be extracted using database connectors or ETL processes.

4. Apache NiFi

Apache NiFi manages and automates the movement of data from different sources into the Hadoop ecosystem.

5. Data Validation

The incoming data is checked before storage to improve data quality.

6. HDFS Data Lake

The processed data is stored centrally in HDFS. This reduces the need to maintain multiple separate copies.

7. Backup and Recovery

Important data is protected using HDFS replication and reliable backup mechanisms.

Justification:

This architecture collects data from multiple enterprise sources through a centralized pipeline. Apache NiFi reduces manual data movement and helps control the ingestion process. Centralized storage in HDFS minimizes unnecessary duplication. Backup and replication mechanisms improve data reliability and recovery.

5. Design an End-to-End Hadoop Ecosystem Solution

Answer:

An end-to-end Hadoop ecosystem solution should provide **secure data movement, high availability, and integration with Apache NiFi or ETL pipelines.**

Proposed Architecture:

Multiple Data Sources



Apache NiFi / ETL Pipeline



Secure Data Movement



HDFS Storage



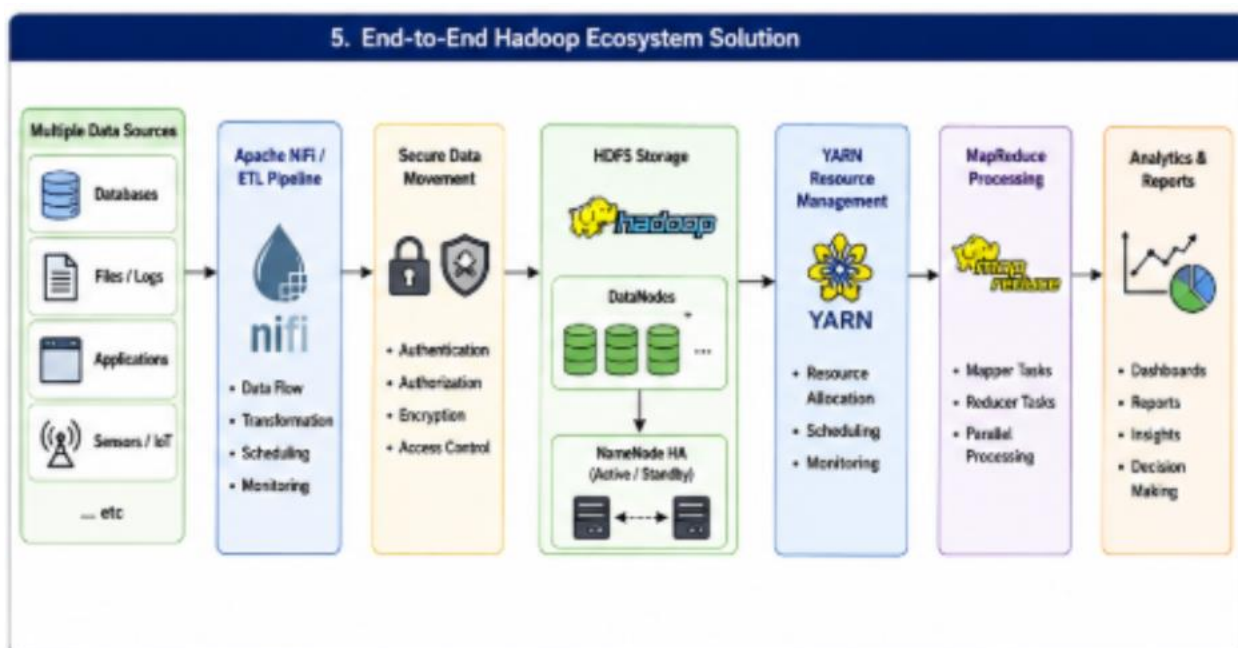
YARN Resource Management



MapReduce Processing



Analytics and Reports



Components and Working:

1. Data Sources

Data can come from databases, files, applications, sensors, and other enterprise systems.

2. Apache NiFi / ETL Pipeline

Apache NiFi collects and transfers data automatically from different sources to the Hadoop platform.

3. Secure Data Movement

Security mechanisms such as authentication, authorization, and controlled access can protect data during movement.

4. HDFS

HDFS stores large amounts of data across multiple DataNodes and provides distributed storage.

5. High Availability

NameNode High Availability can be implemented using Active and Standby NameNodes. This reduces the risk of service interruption.

6. YARN

YARN manages cluster resources and allocates CPU and memory to different applications.

7. MapReduce

MapReduce processes large datasets in parallel using Mapper and Reducer tasks.

8. Analytics and Reports

The processed results can be used for analytics, reporting, and business decision-making.

Justification:

This end-to-end solution satisfies all the required constraints. Apache NiFi provides integration with data ingestion and ETL pipelines. Secure data movement protects sensitive information. HDFS replication and NameNode High Availability provide high availability. YARN manages resources efficiently, and MapReduce performs distributed processing. Therefore, the proposed architecture provides a **secure, scalable, reliable, and complete Hadoop ecosystem solution.**